



field guide · flight control · motor drive · rf links · power · 6 projects · aug 2026

> build six things that would actually fly – then show the scope trace that proves it._

Six EE Projects That Put You on a Drone Team

A flight controller from an IMU and a PID loop. An ESC you built from six MOSFETs. A telemetry link that survives 2 km of open air. A gimbal that holds a horizon, a power module that reports every cell, and an indoor positioning grid for when GPS is gone. Six benches, six flight-ready builds, one portfolio a drone company will actually read.

01	Flight controller from scratch write your own attitude filter and PID loop on an ESP32 + MPU6050 and balance a one-axis rig before you ever spin four props.	\$18
02	Brushless ESC you built drive a 3-phase motor with six MOSFETs and back-EMF commutation on a board you laid out yourself – the part every drone company buys in.	\$14
03	LoRa telemetry link stream GPS, altitude and battery voltage from the airframe to a ground station 2 km away on a 915 MHz LoRa link you budgeted yourself.	\$22
04	Brushless camera gimbal hold a camera level through pitch and roll with two gimbal motors, an IMU on the camera plate and your own sine-wave drive.	\$26
05	Smart battery power module read every cell voltage, pack current and mAh used on a 4S LiPo with your own shunt amplifier, and warn the pilot before the pack sags.	\$16
06	GPS-denied indoor positioning place three UWB anchors around a room and trilaterate a tag on the drone to ± 10 cm – how warehouse drones fly where GPS can't reach.	\$48

● By Enes · @1nska · Electrical Engineer – a Build List field guide

Flight controller from scratch

write your own attitude filter and PID loop on an ESP32 + MPU6050 and balance a one-axis rig before you ever spin four props.

\$18

intermediate

2 weekends

why this matters

Every drone company assumes a graduate has only ever flashed Betaflight, and that assumption is usually correct. This project forces you through the part the firmware hides: raw gyro rates and accelerometer vectors coming in at 1 kHz, a complementary (or Kalman) filter turning them into a roll and pitch angle, and a PID loop turning that angle error into motor commands. You will discover that the textbook loop oscillates on a real rig, that the gyro drifts, that the accelerometer lies under vibration, and that the difference between a stable hover and a flip is a derivative term and a 100 Hz low-pass filter. Debugging that on a hinged one-axis test stand, with a logged plot of setpoint versus measured angle, is the actual skill a flight-controls engineer is hired for.

WHAT THIS PROVES TO A RECRUITER

You can explain where your angle estimate comes from, why you fused gyro and accelerometer instead of trusting either, what your loop rate is and why, and what the D-term does when the motor response is slow. You tuned gains on real hardware with a logged step response, not a slider in a simulator. That is a control-systems story a drone company can check in one question – and almost no one walks in with it.

→ **the job this maps to:** Flight controls / GNC engineer – attitude estimation and inner-loop stabilization for multirotors.

SAFETY WARNING

Run every test with propellers removed until the rig is stable on the stand; a loop that saturates will spin motors to full throttle without warning.

how it works

- ▶ **Two sensors, two failure modes:** the gyro gives clean rate but integrates into drift; the accelerometer gives an absolute tilt but is swamped by motor vibration – the filter trusts the gyro fast and the accelerometer slow.
- ▶ **Loop rate sets the ceiling:** a 250 Hz loop can stabilize a rig whose dynamics are 10–20 Hz; go slower and the phase lag alone will make a well-tuned loop oscillate.
- ▶ **Derivative on measurement, not error:** putting D on the error makes every setpoint step a spike to the motors; on the measurement it only reacts to what the rig is actually doing.
- ▶ **Motor response is part of the plant:** a brushless motor plus ESC has 30–80 ms of lag, which is why the same gains that work in simulation oscillate on the bench.
- ▶ **A test stand is not a cheat:** every real flight-controller team tunes inner loops on a constrained rig first – crashing a rig costs nothing, crashing an airframe costs a week.

key design decisions

- **Why a complementary filter instead of a full Kalman filter?:** at this sensor quality the complementary filter is within a degree of an EKF on roll and pitch and fits in ten lines you can defend at a whiteboard; the EKF is the right answer once you add a magnetometer and GPS.
- **Why an ESP32 instead of an Arduino Uno?:** the Uno's 16 MHz 8-bit core struggles to hold 250 Hz with floating-point math; the ESP32 runs the loop at 1 kHz with headroom and has a second core for logging.
- **Why a one-axis rig instead of tethering the quad?:** a tether still lets the aircraft couple roll, pitch and yaw, so you cannot tell which gain is wrong; one axis, one loop, one plot.

bill of materials

component	part / spec	qty	~cost
Microcontroller	ESP32 DevKit-C	1	\$5
IMU	MPU6050 breakout (GY-521)	1	\$2
Motors + ESCs	2× 2204 brushless + 2× 20 A ESC	1 set	\$8
Test stand	3D-printed cradle + 2 mm carbon rod + bearings	1	\$2
Power	3S LiPo 1000 mAh (props off)	1	\$1 (reuse)

build steps

- Step 1**

Wire the MPU6050 to the ESP32 over I2C at 400 kHz, set the gyro to ± 500 °/s and accel to ± 4 g, and log raw readings at 1 kHz to confirm you actually get 1000 samples a second before you write any control code.
- Build the one-axis test stand**

a 2 mm carbon rod through a 3D-printed cradle holding two motor arms, so the rig can only roll – one degree of freedom means one PID loop to tune, not three.
- Step 3**

Implement the complementary filter ($\text{angle} = 0.98 \cdot (\text{angle} + \text{gyro} \cdot \text{dt}) + 0.02 \cdot \text{accel_angle}$) and plot the output against a protractor at 0°, 15° and 30° before trusting it.
- Step 4**

Write the PID loop at 250 Hz with the D-term on the measurement (not the error) and a 100 Hz first-order filter on it, then tune P first until the rig just oscillates, add D until it settles, add I last to remove the standing offset.

5 Step 5

Log setpoint, measured angle and motor output to a CSV over serial, do a 20° step and keep the plot – rise time, overshoot and settling time are the numbers you will be asked for.

interview questions

- Why can't you just integrate the gyro to get the angle?
- What loop rate did you run and how did you choose it?
- What happens to your loop if the ESC adds 50 ms of lag?
- Why is the D-term on the measurement instead of the error?
- Show me the step response – what is the overshoot and why?

Brushless ESC you built

drive a 3-phase motor with six MOSFETs and back-EMF commutation on a board you laid out yourself – the part every drone company buys in.

\$14

advanced

2 weekends

why this matters

An ESC is a black box to most EE students: a \$12 brick that turns a PWM signal into a spinning motor. Inside it is a three-phase half-bridge, gate drivers, a microcontroller and a way of finding the rotor without sensors – by watching the back-EMF zero crossing on the floating phase. Building it yourself means you will blow a MOSFET by shooting through a half-bridge, you will see the back-EMF on a scope and learn why it only appears once the motor is already turning, and you will write an open-loop start-up ramp that hands off to sensorless commutation. That is the exact circuit inside every motor controller in the industry, and the debugging – dead time, gate charge, current spikes – is the actual job.

WHAT THIS PROVES TO A RECRUITER

You can draw a three-phase bridge from memory, explain dead-time and shoot-through, describe how zero-crossing detection works and why it fails at low speed, and show a scope capture of your own commutation. Motor-drive engineers are among the hardest hires in the drone and EV world; a working board you laid out puts you in a different pile.

→ **the job this maps to:** Motor drive / power electronics engineer – ESC and inverter design for propulsion.

how it works

- ▶ **Six switches, one rule:** at any instant one high-side and one low-side MOSFET on different phases are on, the third phase floats – that floating phase is where you read the back-EMF.
- ▶ **Dead time is not optional:** a high-side and low-side on the same leg overlapping for even 100 ns is a short across the battery; the gate driver's delay plus the MOSFET's turn-off sets it.
- ▶ **Back-EMF needs speed:** the floating phase's voltage is proportional to RPM, so below a few hundred RPM there is nothing to detect – hence the forced open-loop start.
- ▶ **The bootstrap cap is the usual bug:** high-side gates are driven from a capacitor recharged only when the low side is on, so 100 % duty cycle starves it and the high-side drops out.
- ▶ **Current is the thing that kills boards:** a stalled motor draws locked-rotor current; the shunt and a fast comparator trip is what production ESCs do, and why they survive.

BUILD NOTE

Keep a 2 A current limit on the bench supply and a bulk 470 μ F electrolytic right at the bridge – the first spin-up without it will reboot the MCU on the voltage dip.

key design decisions

- **Why sensorless back-EMF instead of Hall sensors?:** drone motors ship without sensors and every commercial ESC is sensorless; Hall sensors are easier but teach you the wrong product.
- **Why discrete gate drivers instead of an integrated 3-phase driver IC?:** the IR2101 makes each half-bridge visible on the scope, so you debug dead-time and bootstrap yourself; integrated drivers like the DRV8302 hide exactly the parts you need to learn.
- **Why a 2-layer board instead of a breadboard?:** 15 A switching at 20 kHz on jumper wires rings, heats and fails; a tight bridge layout with a solid ground pour is the difference between a spike and a working switch node.

bill of materials

component	part / spec	qty	~cost
MOSFETs	6× IRLR7843 (30 V, 3 m, DPAK)	6	\$3
Gate drivers	3× IR2101 half-bridge driver	3	\$2
Microcontroller	ATmega328P or STM32F103 blue pill	1	\$3
Passives	bootstrap diodes, 1 μ F caps, 10 gate R, 1 m shunt	set	\$2
PCB	2-layer, 5 pcs from a board house	1	\$4

build steps

- Design the power stage**
three half-bridges of N-channel MOSFETs (e.g. IRLR7843, 30 V, 3 m) driven by three IR2101 half-bridge gate drivers with a bootstrap cap and 10 gate resistors, and lay it out with the bridge on one side and the MCU on the other.
- Step 2**
Before any motor, test one half-bridge into a resistor load at 12 V with 1 μ s dead time and scope the switch node – a clean edge with no overshoot spike means the gate drive and layout are right.
- Step 3**
Write the six-step commutation table and spin the motor open-loop with a slow forced ramp (like a stepper), props off, at low duty – this proves the bridge and the phase order.
- Add back-EMF sensing**
a resistor divider on each phase into the MCU comparator, detect the zero crossing on the floating phase, wait 30 electrical degrees, then commutate – and log the RPM it locks in at.
- Close the loop**
ramp open-loop for 0.5 s, hand off to sensorless, then add a throttle input from PWM and a current shunt so the board shuts down above 15 A.

interview questions

- Walk me through one commutation step – which switches are on and why?
- What is dead time and what happens if you set it to zero?
- Why can't a sensorless ESC start from standstill in closed loop?
- How does the bootstrap supply fail and how did you avoid it?
- Where did your first board fail and what did you change on the second?

LoRa telemetry link

stream GPS, altitude and battery voltage from the airframe to a ground station 2 km away on a 915 MHz LoRa link you budgeted yourself.

\$22

intermediate

1-2 weekends

why this matters

Every drone beyond toy range needs a telemetry downlink, and every drone company has an RF person who owns it. This project gives you the whole chain: a GPS module and a LoRa radio on the airframe, a matching radio on a ground station, a packet format you designed, and a range test you actually ran. You will learn why LoRa trades bit rate for sensitivity, what a link budget is when you compute it from transmit power, antenna gain and receiver sensitivity, and why the first range test fails at 300 m because the antenna is lying flat against a carbon arm. The logged RSSI-versus-distance plot from a walk test is the kind of evidence an RF hiring manager reads in five seconds.

WHAT THIS PROVES TO A RECRUITER

You can state your link budget with numbers – +20 dBm out, -137 dBm sensitivity at SF12, 2 dBi antennas – and explain what the margin means, why you chose the spreading factor you did, and how you traded packet rate against range. You ran a walk test and have the RSSI plot. That is an RF systems story, not a 'used a LoRa library' story.

→ **the job this maps to:** RF / datalink engineer – telemetry, command-and-control links and range testing for UAS.

how it works

- ▶ **Spreading factor is the knob:** each step up in SF roughly doubles time-on-air and buys ~2.5 dB of sensitivity – SF7 is fast and short-range, SF12 is slow and long-range.
- ▶ **Link budget is arithmetic, not magic:** 20 dBm + 2 dBi + 2 dBi – 97.5 dB (2 km free-space at 915 MHz) = -73.5 dBm at the receiver, against -130 dBm sensitivity – the 56 dB margin is what survives trees, bodies and a bad antenna.
- ▶ **Antenna placement eats margin first:** a whip antenna parallel to a carbon arm is detuned and shadowed; vertical, clear of carbon, on both ends, recovers 10-20 dB for free.
- ▶ **Binary packets, not JSON:** time-on-air scales with bytes, and at SF10 a 20-byte packet is ~330 ms while a 120-byte JSON string is ~1.3 s – you cannot send 2 Hz telemetry in ASCII.
- ▶ **Sequence number plus CRC tells the truth:** packet loss, not RSSI, is what the pilot actually cares about; the logger counts gaps in the sequence.

key design decisions

- **Why LoRa instead of a 2.4 GHz WiFi or nRF24 link?:** at the same transmit power LoRa's processing gain buys 20-30 dB of sensitivity, which is the difference between 200 m and 2 km; WiFi is faster but dies past the car park.

- **Why 915 MHz instead of 433 MHz?:** shorter antennas for the same gain, more bandwidth in the band, and better behaved near the 433 MHz RC-control gear already on many airframes – check your country's ISM band before buying modules.
- **Why a custom binary packet instead of MAVLink?:** MAVLink is what production telemetry uses and you should mention it, but writing your own 20-byte frame teaches you why every byte costs airtime; MAVLink can be dropped in afterwards.

bill of materials

component	part / spec	qty	~cost
LoRa radios	2× SX1276 915 MHz module (RFM95W)	2	\$8
GPS	u-blox NEO-6M with antenna	1	\$5
Microcontrollers	2× ESP32 DevKit-C	2	\$9
Antennas	2× 915 MHz whip, 2 dBi, u.FL/SMA	2	(included)
Misc	5 V BEC, headers, pole for walk test	set	\$0 (scrap)

build steps

1 Step 1

Wire a SX1276 LoRa module (915 MHz, SPI) and a u-blox NEO-6M GPS (UART) to an ESP32 on the air side; power from a 5 V BEC and confirm a GPS fix outdoors before anything else.

2 Step 2

Define a 20-byte binary packet – lat/lon as int32 ×1e7, altitude int16, battery mV uint16, sequence uint8, CRC8 – and write the encoder/decoder with a unit test that round-trips it.

3 Build the ground station

a second SX1276 on an ESP32 that parses packets and streams them over USB serial to a laptop plotting position on a map and RSSI over time.

4 Step 4

Compute the link budget on paper (TX power + antenna gains – path loss – sensitivity = margin) for 2 km free-space at SF10/125 kHz, then pick a spreading factor that gives ≥15 dB margin with ≥2 packets per second.

5 Walk test

log RSSI, SNR and packet loss every 100 m to 2 km with the air unit on a pole, keep the plot, and note where loss starts versus where your budget said it would.

interview questions

- Give me your link budget – where does every dB come from?
- Why did you pick that spreading factor and what did it cost you?
- Where did your range test fall short of the budget and why?
- How do you detect packet loss on the ground station?
- What would you change to reach 10 km?

Brushless camera gimbal

hold a camera level through pitch and roll with two gimbal motors, an IMU on the camera plate and your own sine-wave drive.

\$26

intermediate

2 weekends

why this matters

A gimbal is a control loop you can see working: the camera stays level while you tilt the handle. Under that are two things a drone company cares about – sinusoidal open-loop drive of a brushless motor (three PWM sine waves 120° apart, no commutation sensors) and a fast PID loop closing on an IMU mounted on the camera plate itself. You will learn why the IMU must be on the payload and not the frame, why a 400 Hz loop with a 20 Hz plant still jitters if your PWM frequency is audible, and why the motor gets hot at standstill. The result is a two-axis rig holding a horizon in a video you can post, plus the logged angle error that proves it.

WHAT THIS PROVES TO A RECRUITER

You can explain sinusoidal commutation versus six-step, why a gimbal motor has high pole count and high resistance, and how the loop behaves when the IMU is on the frame instead of the camera. You have a video of it holding level and a plot of residual error. That is a payload/stabilization story – and every inspection, mapping and cinema drone has one of these on it.

→ **the job this maps to:** Payload / stabilization engineer – gimbal drive and camera stabilization systems for UAS.

how it works

- ▶ **Sinusoidal drive, no feedback needed:** three phases fed $\sin()$, $\sin(+120^\circ)$, $\sin(+240^\circ)$ create a rotating field; the rotor follows, so the electrical angle you command is the mechanical position you get.
- ▶ **Gimbal motors are deliberately weak:** many turns of thin wire give high torque per amp at near-zero speed and let you run them at a few hundred mA continuously without burning out.
- ▶ **IMU on the payload, always:** the loop must measure what you are stabilizing; an IMU on the frame tells you the handle moved, not whether the camera did.
- ▶ **PWM above hearing:** at 8 kHz the motor sings; at 20–32 kHz it is silent and the current ripple is lower – pick the frequency your MCU timer can actually hit.
- ▶ **Two axes, two loops, one coupling:** large roll angles rotate the pitch axis, so a 3-axis gimbal needs proper rotation math – two axes is where you learn it without the math becoming the whole project.

BUILD NOTE

Gimbal motors draw constant current at standstill – if the driver runs hot, lower the drive voltage or sine amplitude rather than adding a heatsink; 40 % amplitude holds most small cameras.

key design decisions

- **Why open-loop sinusoidal drive instead of a sensed servo?:** a hobby servo has 1° of dead band and cogs; sine-driven brushless is smooth to a fraction of a degree and is what every camera gimbal on the market uses.
- **Why an L6234 instead of three discrete half-bridges?:** at under 1 A the integrated driver is cheaper and cleaner and lets you spend the project on the control loop – you already built the discrete bridge in the ESC project.
- **Why a 400 Hz loop for a 20 Hz plant?:** the motor field is set instantly but the camera plate has inertia; $20\times$ oversampling keeps the phase lag out of the loop and lets the D-term do its job.

bill of materials

component	part / spec	qty	~cost
Gimbal motors	2× 2208 high-turn brushless gimbal motor	2	\$12
Driver	L6234 three-phase driver breakout	2	\$6
IMU	MPU6050 breakout	1	\$2
Microcontroller	STM32F103 blue pill or ESP32	1	\$3
Frame	3D-printed two-axis arms + M3 hardware	1	\$3

build steps

- Step 1**
Mount two gimbal motors (e.g. 2208, 90 turns, $\sim 10^\circ$) in a two-axis frame – pitch motor carries the camera plate, roll motor carries the pitch motor – and bolt an MPU6050 to the camera plate, not the handle.
- Step 2**
Drive each motor with three PWM outputs through an L6234 (or three half-bridges) at 20–32 kHz, generate sine/cosine tables, and verify the motor turns smoothly when you step the electrical angle – this is open-loop FOC-style drive.
- Step 3**
Read the IMU at 500 Hz, run the complementary filter from the flight-controller project, and get a stable pitch/roll estimate on the camera plate before closing anything.
- Close a PID loop per axis at 400 Hz**
the error is the camera angle versus level, the output is the electrical angle advance of that motor; tune P until it holds, D until it stops humming, I last.
- Step 5**
Record a video tilting the handle $\pm 30^\circ$ while logging camera angle error; the goal is under $\pm 0.5^\circ$ steady-state and no visible bounce on a step.

interview questions

- How does a brushless motor hold position without a position sensor here?
- Why must the IMU be on the camera plate?
- What PWM frequency did you use and why?
- What limited your steady-state error to $\pm 0.5^\circ$?
- How would you extend this to three axes?

Smart battery power module

read every cell voltage, pack current and mAh used on a 4S LiPo with your own shunt amplifier, and warn the pilot before the pack sags.

\$16

beginner

1 weekend

why this matters

Batteries decide flight time and batteries cause crashes, so every drone company has someone who owns the power module. This project is that module: a shunt resistor and current-sense amplifier for pack current, a resistor-divider per cell into a multiplexed ADC for cell voltages, and firmware that integrates current into mAh consumed and flags a weak cell. You will learn why a 0.5 m shunt needs a 50 V/V amplifier, why the divider ratio matters for ADC resolution, why cell voltage under load and at rest are different numbers, and why mAh consumed is a better land-now signal than voltage. The logged discharge curve you produce is the first thing a battery engineer asks for.

WHAT THIS PROVES TO A RECRUITER

You can explain high-side versus low-side current sensing, size a shunt for a 60 A pack, talk about coulomb counting versus voltage estimation, and show a discharge curve with a weak cell visible in it. That is battery-management experience, and 'BMS' shows up in nearly every drone, EV and robotics hardware posting.

→ **the job this maps to:** Battery / power systems engineer – BMS, power monitoring and energy estimation for electric aircraft.

SAFETY WARNING

Never leave a LiPo connected to a discharge load unattended, and stop any test the moment a cell reads under 3.0 V – over-discharged cells are the ones that catch fire on the next charge.

how it works

- ▶ **Shunt plus amplifier is the whole current sensor:** $V = I \cdot R$, and a milliohm shunt keeps the loss to a few watts at full throttle while a dedicated current-sense amplifier rejects the common-mode voltage of the pack.
- ▶ **Cell voltages come from differences:** each balance tap is measured to ground, then cell n is tap n minus tap $n-1$ – which is why ADC resolution and matched dividers matter more the higher the cell.
- ▶ **Voltage lies under load:** a pack at 14.8 V resting can read 13.6 V at full throttle from internal resistance alone, which is why voltage alarms trip early and coulomb counting is what real power modules use.
- ▶ **A weak cell hides at rest:** all four cells read 4.1 V on the bench and one dips to 3.2 V under load; per-cell logging under load is how you find it before it ends a flight.

- ▶ **mAh used is the honest fuel gauge:** integrate current, compare to rated capacity, and land at 80 % – that is exactly the rule ArduPilot and PX4 power modules enforce.

key design decisions

- **Why a dedicated current-sense amplifier instead of an op-amp?:** an INA240 rejects 80 dB of common-mode voltage and PWM switching noise at the shunt; a generic op-amp divider on a 16.8 V rail gives you millivolts of error on a millivolt signal.
- **Why a 16-bit external ADC instead of the MCU's 12-bit ADC?:** 4.2 V per cell through a divider into 12 bits gives ~5 mV steps; the ADS1115 gives sub-millivolt resolution, which is what you need to see a cell drift.
- **Why low-side shunt here?:** it keeps the amplifier referenced to ground and is how most hobby power modules are built; high-side sensing is the production answer when the ground path must stay unbroken – mention both.

bill of materials

component	part / spec	qty	~cost
Shunt	1 m 2512 current-sense resistor, 3 W	2	\$1
Current amplifier	INA240A2 (50 V/V) or INA181 breakout	1	\$3
ADC	ADS1115 16-bit I2C breakout	1	\$3
Microcontroller + display	ESP32 + 0.96-inch OLED + buzzer	1 set	\$7
Connectors	XT60 pair, JST-XH balance lead	set	\$2

build steps

1 Size the shunt

for 60 A peak and a 75 mV full-scale amplifier input, choose 1 m (60 mV at 60 A, 3.6 W peak – use a 2512 current-sense resistor rated 3 W or two in parallel) and an INA240 or INA181 amplifier with 50 V/V gain into the ADC.

2 Build the cell-voltage stage

a resistor divider from each balance-lead tap (1S...4S) to an ADS1115 16-bit ADC, scaled so 4.2 V per cell lands below 3.3 V full scale, and compute per-cell voltage as the difference between adjacent taps.

3 Step 3

Calibrate against a bench meter at 0, 5 and 20 A and at three known voltages – store the slope and offset in flash, not in a comment.

4 Step 4

Integrate current at 100 Hz into mAh consumed, display pack voltage, lowest cell and mAh on a 0.96-inch OLED, and drive a buzzer when mAh used hits 80 % of rated capacity or any cell drops under 3.5 V under load.

5 Step 5

Log a full discharge on a bench load (or a tethered prop-off motor run) at 10 A and keep the curve – pack voltage, each cell, and mAh versus time.

interview questions

- How did you size the shunt and what does it dissipate at full throttle?
- Why is per-cell voltage a difference measurement?
- Why is a voltage alarm a bad fuel gauge?
- What did your discharge curve show that a single voltage reading would not?
- Where would you put the shunt in a production airframe and why?

GPS-denied indoor positioning

place three UWB anchors around a room and trilaterate a tag on the drone to ± 10 cm – how warehouse drones fly where GPS can't reach.

\$48

advanced

2 weekends

why this matters

Indoor drones are the fastest-growing part of the industry – warehouse inventory, tunnel and tank inspection – and none of them have GPS. This project is the answer those teams actually use: ultra-wideband ranging. Three ESP32 + DW1000 anchors on tripods, one tag on the airframe, two-way ranging that times a radio pulse round trip to nanoseconds, and trilateration that turns three distances into an x/y position on a laptop map. You will learn why UWB can range to centimetres when WiFi cannot, why antenna delay calibration is the whole game, and why three anchors give you a position but four give you a sanity check. The logged position trace as you walk a square is proof you can build a localization system, not just call a library.

WHAT THIS PROVES TO A RECRUITER

You can explain two-way ranging and why it cancels clock offset, what limits UWB accuracy indoors (antenna delay, multipath, geometry), how trilateration degrades when anchors are nearly collinear, and you have a plot of a walked square that looks like a square. Localization and sensor fusion are exactly what indoor-drone companies screen for – and this build is the first rung of it.

→ **the job this maps to:** Localization / navigation engineer – GPS-denied positioning and sensor fusion for indoor UAS.

how it works

- ▶ **Time of flight, not signal strength:** UWB sends a 2 ns pulse and measures when it comes back; 1 ns of timing error is 30 cm, which is why the DW1000 timestamps to ~15 ps.
- ▶ **Two-way ranging cancels clocks:** tag sends, anchor replies after a known delay, tag times the round trip – neither side needs a synchronized clock, only a stable one.
- ▶ **Antenna delay is the calibration:** the chip timestamps at the transceiver, but the signal spends extra nanoseconds in the PCB trace and antenna; that fixed delay is subtracted per module and is why uncalibrated modules read 30–50 cm long.
- ▶ **Geometry sets the error:** three anchors in a line cannot tell which side of the line you are on; spread them around the volume and the same ranging error gives a far smaller position error (dilution of precision).
- ▶ **Multipath is the indoor enemy:** a pulse that bounces off a metal shelf arrives late and reads long; the DW1000 reports first-path power so you can reject ranges that came in via a wall.

key design decisions

- **Why UWB instead of WiFi or BLE RSSI positioning?:** RSSI varies 10 dB with a body in the way, which is metres of error; UWB time-of-flight is physics-limited to centimetres and is what every commercial indoor RTLS uses.
- **Why trilaterate on the laptop instead of on the tag?:** you want to see every raw range, residual and rejected measurement while you learn it; moving the solver onto the tag is a one-evening port once it works.
- **Why three anchors first, then four?:** three is the minimum for a 2D fix and forces you to understand the geometry; the fourth anchor introduces redundancy and residuals – the thing that makes a real system trustworthy.

bill of materials

component	part / spec	qty	~cost
UWB modules	4× ESP32 + DW1000 board (Makerfabs ESP32-UWB)	4	\$40
Mounting	3 cheap tripods or clamps + tape measure	set	\$6
Power	USB power banks for anchors	3	\$0 (reuse)
Tag mount	3D-printed clip for the airframe	1	\$1
Cable	USB for the tag's serial link	1	\$1

build steps

1 Step 1

Flash three ESP32 + DW1000 boards (Makerfabs ESP32-UWB or a DWM1000 on a breakout) as anchors at known coordinates – tape measure them to the centimetre, corners of a 4 m × 4 m square, 1.5 m high – and one as a tag.

2 Step 2

Implement (or run) double-sided two-way ranging between the tag and each anchor in turn, and read the raw distance at 1 m, 2 m and 4 m against a tape – expect a fixed offset of tens of centimetres.

3 Calibrate antenna delay

adjust the per-module delay constant until all three anchors read within ±5 cm at a known 2 m – this single number is where most of the error lives.

4 Trilaterate on the laptop

three circles, least-squares intersection (solve the linearized 2×2 system), plot the tag position live over serial at 10 Hz, and watch it jitter when you stand between the tag and an anchor.

5 Step 5

Walk a 2 m square holding the tag and log the trace; overlay it on the true square and report RMS error – then add a fourth anchor and show how the residual tells you when one range is wrong.

interview questions

- Why does two-way ranging not need synchronized clocks?
- What is antenna delay and how did you calibrate it?
- How does anchor geometry change your position error?
- How do you detect a multipath range and what do you do with it?
- How would you fuse this with the IMU from your flight controller?

> wrap

You just built the electronics stack of a real aircraft.

Strip any commercial drone down and it is these six things: an IMU feeding a control loop, three-phase motor drive, a radio link, a stabilized payload, a battery that reports on itself, and a way to know where the aircraft is. Every project here is one of those blocks, built small enough to finish on a bench and real enough to put in an airframe.

None of them are toy demos. Each one maps to a line in a drone-company job posting, and each one leaves you with a log file, a scope capture, a photo of the board, and a failure you diagnosed yourself – the four things a technical interviewer actually wants to see.

// the skill ladder

IMU + control loops → three-phase motor drive → RF link budgets → sensor fusion → battery management → system-level flight test

THE RULES, EVERY SINGLE TIME

Props off for every bench test. A motor that spins up by accident with a propeller on it is the single most common way people get hurt on these builds – remove the props before you power anything.

Keep a current-limited bench supply between you and any new board until you have measured its idle draw; a reversed rail or a shorted MOSFET should pop a limit, not a battery.

Never charge, balance or discharge a LiPo unattended, and never below 3.0 V per cell – a puffed pack goes in a fireproof bag outdoors, not back on the bench.

DO THIS

Build one of these all the way to a logged flight test or bench run, write down the one thing that broke and how you found it, then post the photo and the plot. Comment the keyword on this guide's post and I'll send you the full write-up for free.

The Build List is Enes's running series of these field guides – join the Ohm's Club to get the next one the day it ships.

– Enes (@1nska) · Electrical Engineer